STUDENT READING NOTES

WEEK 01 OF 16

CodeWithProf

NextGen Web, Cloud & AI Engineering Bootcamp

(Explorers) Cohort 1

A 16-week learning journey for ages 9-14

Welcome to Technology & How the Internet Works

> **BIG IDEA**
>
> Every app is built by people. When you use an online app, computers exchange requests and responses.

How to use this handbook

- Read one section at a time and explain the idea in your own words.
- Follow the fictional StudyHub example through each topic.
- Use the Week 01 lab and assignment for hands-on practice.
- Builder and Career Launch notes are optional extensions.

Your Week 01 learning map

Part	What you will understand
1. Software engineering	What software is, how teams build it and the different careers involved
2. The internet	The difference between the internet and the web; clients and servers
3. Finding a website	IP addresses, domain names, DNS and URL structure
4. Web conversations	HTTP, HTTPS, requests, responses, status codes and APIs
5. The bigger system	Cloud computing, databases and production architecture
6. Responsible technology	Online safety, privacy and thoughtful use of AI

Meet StudyHub

Imagine a learning website called **StudyHub**. Students sign in to see their courses, upload homework and receive AI-supported revision suggestions. Teachers add assignments and mark work. StudyHub is fictional, so we can safely use made-up account details and examples.

StudyHub is not just one screen. It needs a user-friendly interface, a server that processes requests, a place to store information and people who operate and protect the system. Throughout Week 01, imagine what happens when a student clicks **Submit homework**.

> **Guiding question:** Who designs that button? Who receives the homework? Where does it go? How does the student know the upload succeeded?

1. What is software engineering?

Software means instructions that tell a computer what to do. A calculator app, a video game, a browser and the controls inside a washing machine all rely on software. **Programming** is writing code that expresses these instructions. **Software engineering** goes further: it is the planned process of building, testing, delivering, maintaining and improving useful software.

Think of building a bridge. The engineer cannot simply place materials anywhere and hope they hold. A safe bridge requires planning, calculations, tests and maintenance. Similarly, an app that works once is not automatically a good product. It should work reliably, protect information, and be understandable to the team that will improve it later.

The software engineering cycle

Plan → Design → Build → Test → Release → Improve

A team plans the problem it wants to solve, designs an approach, builds features, tests them, releases software to users and learns from feedback. Then it repeats. This is not always a straight line: teams may go back to design when tests reveal a problem.

Example: a homework upload feature

- **Plan:** Students need a safe way to submit assignments before a deadline.
- **Design:** Decide which buttons, messages and data the feature needs.
- **Build:** Write the frontend and backend code.
- **Test:** Try a valid file, a file that is too large and a missing file.
- **Release:** Make the feature available to students.
- **Improve:** Read feedback, fix bugs and make the experience clearer.

> **Try thinking:** Why is “the button works on my laptop” not enough proof that the upload feature is ready for a whole school?

2. What do software engineers do all day?

A software engineer does far more than type code. Teams discuss user problems, break work into small tasks, read existing software, review changes, test releases, investigate bugs and write documentation. Explaining decisions clearly is an engineering skill.

A typical working day

A developer might begin with a short team check-in (often called a **stand-up**), choose a task, investigate how the app currently works, make a small change, test it and ask another engineer to review it. A review is an opportunity to catch problems and share understanding—not just to find mistakes.

Career Launch: In companies, teams also make trade-offs between cost, speed, security, reliability and maintainability. They often use branches, pull requests, automated tests and deployment pipelines. We will practise these ideas later in the programme.

3. The technology team: where could you fit?

Technology has space for designers, problem-solvers, investigators, communicators and people who enjoy organising work. A full-stack developer works across the parts users see and the server-side parts users do not see.

Role	Main responsibility	In StudyHub
Frontend	Builds screens and interactions	The course page and upload button
Backend	Builds server logic and APIs	Checks login and saves submissions
Full stack	Works across frontend and backend	Delivers the complete upload feature
UI/UX design	Makes products clear and usable	Designs the submission journey
QA / testing	Checks for errors and unexpected behaviour	Tests uploads and deadlines
Cloud / DevOps	Runs systems and automates safe releases	Hosts and deploys StudyHub
Cybersecurity	Protects systems and user information	Keeps accounts and grades private
Data / AI	Works with data and intelligent features	Analyses progress; revision helper
Product management	Chooses what problem to solve next	Prioritises teacher and student needs

These roles overlap and vary by employer. A small company may have one person doing several jobs; larger companies may have specialised teams. The goal of Week 01 is not to choose your future job, but to notice which problems interest you.

> **Try thinking:** If the StudyHub upload is slow, which roles might need to work together? There can be more than one correct answer.

4. The internet is not the same as the web

The **internet** is a global network of networks connecting computers and other devices. It includes physical links such as fibre-optic cables, wireless connections, routers and other equipment that carries information. The **World Wide Web** is a service that uses this network to deliver linked documents and applications that we open in browsers.

Imagine the internet as roads connecting cities, while the web is one of the services using those roads. Email, online games and video calls also use the internet, but they are not necessarily web pages. The World Wide Web was proposed by Tim Berners-Lee at CERN in 1989.

Clients, servers, requests and responses

A **client** asks for something. Your web browser is a common client. A **server** is a computer program, running on a machine, that receives requests and sends responses. A server can return a webpage, an image, a message or data.

Browser (client) → HTTP request → Server → HTTP response → Browser displays

When a student opens a homework page, the browser requests a resource from StudyHub. The server checks the request and returns the information needed to show it. Some requests may require the server to consult a database first.

> **Remember:** “Client” and “server” describe roles in a conversation, not necessarily a particular shape or size of computer.

5. How does a browser find the right server?

IP addresses: numerical network addresses

Computers use **IP addresses** to route information. One example IPv4 address is **203.0.113.10**; one example IPv6 address is **2001:db8::1**. These are example addresses intended for documentation, not real StudyHub servers. IPv4 uses 32 bits, while IPv6 uses 128 bits, providing a much larger address space.

You may also see **127.0.0.1** or **::1**, the loopback addresses for **localhost**. These refer back to your own computer, which becomes useful when we run a development server later in the course.

Domains and DNS: names humans can remember

A **domain name** is a readable name such as **example.com**. **DNS** (Domain Name System) helps a device find the network address associated with the name. It works like a directory: you know the name, and DNS helps locate the address.

A simplified lookup is: (1) you enter a domain; (2) a DNS resolver looks for an answer, sometimes from its cache; (3) if necessary, the DNS system finds the appropriate records; (4) the resolver returns address information; (5) your browser can connect to the destination.

For Builders: where the answer comes from

A resolver that lacks a cached answer may consult root name servers, top-level domain servers and the domain’s authoritative name servers. Cached answers save work on future lookups. In a terminal, you can inspect a lookup using **nslookup example.com**. Different networks or devices may return different valid addresses.

For Career Launch: useful DNS records

Record	What it contains
A	A domain name mapped to an IPv4 address
AAAA	A domain name mapped to an IPv6 address
CNAME	An alias pointing one name to another name
MX	Mail-routing information for a domain
TXT	Text records, often used for verification and email policy
TTL	A cache lifetime associated with a DNS record

Think like an engineer: A website appears to be down. Is its server failing—or is the name failing to resolve? DNS is one useful early check.

6. Reading a web address (URL)

A **URL** (Uniform Resource Locator) tells a browser where a resource is and how to access it. Consider this fictional example:

https://www.studyhub.example:443/courses/maths?level=beginner&page=2#reviews

Part	Example	Meaning
Scheme	https	Use HTTPS to communicate
Host / domain	www.studyhub.example	The destination host name
Port	443	The connection’s service port
Path	/courses/maths	The requested resource route
Query	level=beginner&page=2	Extra parameters in name=value form
Fragment	#reviews	Points to a location in the page

Most browsers hide default ports such as 443 for HTTPS and 80 for HTTP. A fragment beginning with **#** is handled by the browser and is not sent in the HTTP request target. A path and query string can help the server decide what to return.

A carefully written URL also matters for safety. A fake site can use a name that looks almost identical to a trusted site, so read the hostname instead of relying on the page's appearance.

7. HTTP: the language of web requests

HTTP stands for Hypertext Transfer Protocol. It describes the request-and-response messages that web clients and servers exchange. A request usually contains a **method**, a target path, headers and sometimes a body. A response contains a **status code**, headers and often a body.

Common HTTP methods

Method	Typical purpose	Example
GET	Read information	Get available homework
POST	Submit or create something	Upload a new submission
PUT / PATCH	Replace or update something	Change homework details
DELETE	Remove something	Delete a draft submission

What status codes tell us

Family	Meaning	Familiar example
2xx	Successful request	200 OK
3xx	Redirect or other redirection response	301 Moved Permanently
4xx	Client-side request problem	404 Not Found; 401 unauthenticated; 403 forbidden
5xx	Server-side error	500 Internal Server Error

A status code is not the full story. Developers also inspect response headers and the returned content. If StudyHub shows an error, a developer can open **Developer Tools → Network** in a browser to see requests, responses, timing and status codes.

HTTPS adds an encrypted connection

HTTPS uses TLS to encrypt information in transit and authenticate the server for the domain being contacted. It helps prevent people on the network from reading or changing the conversation. Importantly, a padlock or HTTPS indicator does **not** guarantee that a website or business is trustworthy. Scammers can also use HTTPS.

Career Launch: Networks have layers. IP moves packets between networks. TCP is often used for reliable delivery; HTTP/3 instead uses QUIC over UDP. TLS secures the communication. HTTP defines the web-level messages. HTTP itself is stateless, so applications need additional mechanisms such as cookies or tokens to maintain login state.

8. What is an API?

API means Application Programming Interface. It is a defined way for one program to interact with another. Many web APIs return structured data rather than finished webpages. A common format is **JSON**, which is designed to be easy for software to read.

> **Example JSON response:**
> {"course": "Maths", "homeworkDue": "Friday", "submitted": false}

Imagine StudyHub's frontend asks the backend for the current student's homework list. The backend sends data, and the frontend decides how to display it. This is why frontend and backend developers must agree on what an API expects and returns.

Real-world examples include weather services, maps and payment systems. During the course you will learn to call APIs, then build an API yourself. Never assume an API is safe to expose private data merely because its output looks simple.

9. Databases and cloud computing

An application often needs to remember things after a user closes the page. A **database** stores and organises information so software can retrieve and update it. StudyHub might store users, courses, deadlines and submission records. A file upload might also place the actual file in object storage, while keeping its details in a database.

The **cloud** is not a floating computer in the sky. Cloud computing means using computing services delivered over a network from infrastructure operated by a provider. Companies rent servers, databases, storage and other services instead of necessarily buying and maintaining all the equipment themselves.

Cloud service models for Career Launch

Model	What you get	Example
IaaS	Virtual machines; you manage much of the system	Azure Virtual Machines
PaaS	A platform to run your application code	Azure App Service
SaaS	A completed application you use	Microsoft 365

Cloud systems can grow or shrink as demand changes, but they still need thoughtful security, monitoring and cost management. Cloud infrastructure does not automatically solve bad application design.

10. Follow the full journey: opening StudyHub

Now combine the ideas. A student types the StudyHub address and opens the homework page.

Browser → DNS lookup → Web server → App / API → Database

- **1.** The student enters a URL. The browser identifies its host name.
- **2.** DNS helps the browser find an IP address for the destination.
- **3.** The browser establishes a connection and uses HTTPS to communicate securely.
- **4.** It sends an HTTP request for the homework resource.
- **5.** The application checks what is requested and may read information from a database.
- **6.** The server responds with a status code and webpage content or API data.
- **7.** The browser displays the result and may make more requests for styles, images and JavaScript.

The actual process can include caches, redirects, multiple servers and many separate resources. This is a teaching model showing the most important ideas first.

For Career Launch: how large systems evolve

A busy production application may use a **CDN** to deliver images, stylesheets and other static resources closer to users. A **load balancer** shares application traffic across several healthy backend instances. The application may store data in a managed database. If one server becomes unhealthy, traffic can sometimes be directed to another.

Browser → CDN / entry → Load balancer → App servers → Database

Every extra component adds a responsibility. Architects ask: What happens when this part fails? Where is user data stored? How will we detect problems? What can we make faster without making it less secure?

11. Responsible technology: protect people

Good online safety habits

- Use long, unique passwords or passphrases. Where possible, use a password manager and enable multi-factor authentication.
- Watch for phishing: urgent messages, suspicious links and lookalike addresses are warning signs.
- Do not share private details in class examples, public repositories or screenshots. Use fictional names and test data in this programme.
- If something online makes you uncomfortable, contact a trusted adult or instructor.
- Only investigate or test security on systems you own or have explicit permission to test.

Responsible use of AI

Artificial intelligence includes recommendation systems, spam filtering, image recognition and text-generating tools. AI can help explain unfamiliar topics and suggest ideas, but its outputs can be inaccurate, biased or incomplete. A confident-sounding answer is not proof that it is correct.

In this programme, learners under 18 use AI only through instructor-approved access, subject to the applicable service rules. Do not enter private student information or secrets into AI tools. Declare when AI helped with an assignment, check important claims, and make sure you can explain work you submit.

> **Learning rule:** Use AI to strengthen your understanding, not to replace your thinking.

12. Check your understanding

Try answering before looking back at the chapter. Write complete sentences where possible.

1. Explain the difference between programming and software engineering.

2. Name three roles involved in launching a StudyHub homework feature and explain what each contributes.

3. How is the internet different from the World Wide Web?

4. In the client-server model, which side normally sends the request?

5. What do IP addresses and DNS do?

6. Identify the scheme, hostname, path and query in https://example.com/courses?week=1.

7. What does an HTTP 404 response tell you? How is it different from 500?

8. Why is HTTPS important, and why does it not prove a site is honest?

9. What is JSON, and why might StudyHub use it in an API?

10. Walk through what happens after a student opens a webpage, from DNS through rendering.

11. How are a database, a cloud server and an application different?

12. List two safe practices when using AI for homework.

Discussion questions

- A student says, “The cloud means the website is stored in the air.” How would you explain it kindly?
- StudyHub’s page loads, but its homework list is empty. What different parts of the system might be involved?
- A page has a padlock icon but asks for your password through a suspicious link. What should you do?

13. Your Week 01 quick glossary

Term	Meaning
Software	Instructions used by a computer or device
Software engineering	Planning, building, testing and improving software
Frontend / backend	User-facing interface / application and data processing behind it
Full stack	Working across frontend and backend
Internet / web	Connected networks / a linked-document and application service on those networks
Client / server	The requester / the responder in a network exchange
DNS / IP address	Name-resolution system / numeric network address
URL	The address of a resource, including scheme and location
HTTP / HTTPS	Web communication protocol / HTTP secured with TLS
API / JSON	Program interaction interface / structured data format
Status code	Number describing the result of an HTTP request
Database	A system for organising and retrieving stored information
Cloud computing	Computing services provided over a network
CDN / load balancer	Distributed content delivery / traffic distribution across services
Phishing	Deceptive attempts to get data or prompt harmful actions

Where we go next

In **Week 02**, you will begin writing **HTML**, the markup language browsers use to structure web pages. That means you will create one of the very things a server can send to a browser. Your first personal web page will turn Week 01's internet conversation into something you made yourself.

> **Your challenge:** Be able to explain “Browser → Internet → Server → Database → Response” to someone who has never written a line of code.

Keep learning. Keep building. | codewithprof.com